

# 核心算法与复杂度附录

面向复杂任务求解的多智能体动态协作与自治交付系统

西安交通大学 代码核对日期：2026 年 9 月 14 日

本附录给出五项核心机制的工程化伪代码、代码入口及复杂度边界。算法独立编号为 A1–A5，分别对应主报告第三至第六章；不新增实验结果。伪代码是多个实现函数的摘要，不是可直接执行的程序。符号与保守上界统一见第6节。

## 1 需求前沿驱动的版本化任务图扩展

**作用** 将尚未完成且依赖已满足的需求转化为下一阶段子图，保留已有节点的执行结果；阶段数目本身不作为“任务成功”的依据。

---

### 算法 A1: 需求前沿选择、阶段规划与受控合并

---

**输入:** 当前图  $G$ ，需求进度  $F$ ，全局任务契约  $T$ ，阶段预算  $b$

**输出:** 扩展图  $G^+$ ，或保留原图并记录阻塞原因

- 1 从  $G$  刷新需求进度并持久化  $F$ ;
  - 2 **if** 存在失败或阻塞的需求生产节点 **then**
  - 3   | 记录暂停原因; **return** 不扩展;
  - 4 **if** 不存在待规划需求 **then**
  - 5   | 按需追加一次全局治理节点; **return** 当前或治理后的图;
  - 6 根据需求依赖、批量上限和提示预算选择就绪需求集合  $F_b$ ;
  - 7 **if**  $F_b = \emptyset$  **then**
  - 8   | 记录需求依赖阻塞; **return** 不扩展;
  - 9 调用阶段规划器生成候选; 编译为阶段 TaskGraph  $\Delta G$ ;
  - 10 为新增节点和内部引用添加阶段命名空间;
  - 11 检查物理产物写入归属、全局产物契约及代码节点限制;
  - 12 将新增根节点接到原图中非治理叶节点之后;
  - 13 构造  $G^+$ : 保留原节点状态, 追加新节点, 递增图版本;
  - 14 检查节点 ID、依赖存在性、DAG、能力与产物所有权;
  - 15 **if** 阶段规划或合法性检查失败 **then**
  - 16   | 保留已完成节点; 记录可恢复的阶段阻塞及失败原因;
  - 17 **else**
  - 18   | 保存  $G^+$ 、需求生产者映射和阶段事件;
- 

**实现入口** `orchestration/core/workflow.py: expand_requirement_frontier`; `orchestration/planning/staged_expansion.py: merge_stage_graph`, `append_governance_nodes`; `orchestration/graph/task_graph.py: validate_graph`.

**不变量与边界** 扩展不清空已完成节点，不允许新增节点越过全局物理产物写入契约。当前是阶段前沿扩展，不宣称能在任意执行指令处并发改图。规划异常可留下阻塞证据，但不会因此直接产生成功交付。

**复杂度口径** 合并后的图含  $n$  个节点、 $m$  条依赖边。节点复制和依赖连接属于控制面开销，完整扩展还包含需求处理、契约检查和全图合法性检查，不能仅按新增节点数计费；模型生成时间另计。回归入口: `tests/test_requirement_staging.py`。

## 2 稀疏依赖上下文与受保护记忆构造

**作用** 依赖消息与运行期记忆走两条互补路径：前者只读取直接依赖，后者按相关性检索历史记录；目标、约束和结构化证据不交给有损摘要任意改写。

---

### 算法 A2: 直接依赖消息与 ContextCapsule 的组合构造

---

**输入:** 图  $G$ , 当前节点  $v$ , 任务契约  $T$ , 运行记忆  $\mathcal{M}$ , 通信与记忆预算

**输出:** 节点执行上下文, 或上下文构造异常

- 1 从图中取得规范节点与其直接依赖结果;
  - 2 **foreach** 直接依赖结果 **do**
  - 3     投影到消息字段白名单, 保留声明的必需字段;
  - 4     对普通文本限额压缩; 对大结构化值或重复内容生成带哈希的内容引用;
  - 5 检查单消息及节点依赖上下文预算, 必要时进一步外置内容;
  - 6 **if** 受保护字段或引用元数据仍无法满足硬预算 **then**
  - 7     抛出预算异常, 禁止交付该上下文;
  - 8 读取本轮有效记忆, 分离核心目标约束与普通候选;
  - 9 构建节点索引, 逐条计算相关性、时效、重要性、图邻近度和证据质量;
  - 10 排序普通候选, 取配置允许的前  $k$  条;
  - 11 压缩所选普通摘要; 将全局目标、关键约束、图位置和引用独立放入 capsule;
  - 12 **if** 目标为空或关键约束数量校验不通过 **then**
  - 13     抛出受保护字段异常;
  - 14 保存 capsule 与通信指标, 组合为执行器输入;
- 

**实现入口** `orchestration/communication/context_builder.py: ContextBuilder`; `orchestration/memory/runtime_memory.py: build_capsule、_score`; `orchestration/graph/graph_scheduler.py: _context_for`。

**接口边界** 依赖消息预算与记忆摘要预算分别实施。完整模型提示还由 `orchestration/core/prompt_builder.py` 组装和检查, 不能仅凭某一摘要满足预算, 就宣称整个请求一定满足模型窗口。内容引用有哈希校验; 当前 capsule 保存为结构化 JSON, 不将所有字段都描述成统一签名对象。

**复杂度** 设有效记忆条数为  $R$ 、记忆总内容规模为  $S_R$ , 一次评分涉及的祖先遍历工作量为  $a_v$ , 其最坏上界为  $O(n + m)$ 。候选检索、评分与排序的控制开销可保守写为:

$$T_{\text{memory}} = O(n + S_R + R a_v + R \log(R + 1)).$$

SQLite 查询、有损压缩和上下文序列化另计; 节点索引、已加载记录和排序数组都会占用内存, 不能把空间仅写成输出的  $k$  条记录。仅当单条评分和记录长度有界时, 候选处理才可简化为  $O(R \log R)$ 。

**可核查产物** 依赖内容引用、通信指标及 `memory/capsules/` 下的节点上下文。测试入口: `tests/test_communication.py`、`tests/test_runtime_memory.py`、`tests/test_prompt_budget_semantics.py`。

### 3 受保护任务契约下的 Checkpoint 恢复

**作用** 在已持久化的执行窗口边界恢复任务图和进度。完整性校验、目标一致性与完成状态共同约束恢复，而非重新解释用户需求。

---

#### 算法 A3: 窗口级 checkpoint 的验证与继续执行

---

**输入:** 运行目录, checkpoint 引用, 预期目标和 TaskSpec

**输出:** 恢复后的任务图及窗口状态, 或拒绝恢复

- 1 检查显式引用不是绝对路径且不含上级目录跳转; 未指定时选最新 checkpoint;
  - 2 读取 JSON; 重新计算去除 integrity 字段后的规范化内容哈希;
  - 3 **if** 文件缺失、摘要不符或缺少完整任务图 **then**
  - 4     拒绝恢复并报告原因;
  - 5 以 TaskGraph 模型重新验证图结构和契约字段;
  - 6 **if** 保护目标与图目标不一致 **then**
  - 7     拒绝恢复;
  - 8 **if** 调用方提供预期目标或 TaskSpec **then**
  - 9     对比目标、必需产物列表与受保护任务契约摘要;
  - 10     **if** 任一比较失败 **then**
  - 11         拒绝恢复, 不自动复用不一致的旧状态;
  - 12 恢复图与 epoch, 登记已完成节点集合并记录恢复事件;
  - 13 按当前依赖状态选择后续 ready 窗口, 已完成节点不重新进入该队列;
  - 14 窗口结束后保存任务图、节点状态、受保护上下文和新内容哈希;
  - 15 由正常执行与验收流程决定是否完成, 而非仅检查 epoch 数量;
- 

**实现入口** orchestration/core/workflow.py。主要函数: load\_horizon\_checkpoint、\_execute\_graph\_in\_horizons、\_protected\_task\_digest。

**保护范围** 主流程恢复时传入预期目标与任务契约。当前契约摘要覆盖任务 ID、任务类型、必需产物、成功条件和报告章节等字段, 不应称为对整个 TaskSpec 任意字段的全量校验。该入口也不逐一重算所有外部 artifact 的哈希; 产物检查由执行与验收路径分别承担。

**恢复保证** 已落盘的 completed 节点可以在继续执行时跳过; 这不是任意崩溃时刻的 exactly-once 保证, 也不恢复浏览器会话、外部模型内部状态或全部内存对象。SHA256 是内容完整性摘要, 不是带身份认证的数字签名。

**复杂度** 令 checkpoint 序列化字节数为  $S_K$ , 图检查成本为  $T_{\text{graph}}$ 。读取、解析和哈希的基础开销为  $O(S_K)$ , 恢复还需加上  $T_{\text{graph}}$ 。当前规范化 JSON 使用键排序, 其额外比较工作量另记为  $T_{\text{canon}}$ , 不将其无条件忽略。未指定引用时还需枚举并排序  $K$  个 checkpoint, 成本至多  $O(K \log(K+1))$ 。内存至少覆盖解析后的快照规模。

**业务长程的独立实现** 城市千步实验使用 task\_plugins/urban\_multimodal/long\_horizon.py 中的 WorkUnitLedger.resume\_at, 校验 checkpoint、账本长度、目标摘要和末记录摘要; 完整链验证另由业务验证器完成。该实现与通用任务图恢复不可混为一个入口。测试入口: tests/test\_long\_horizon\_continuity.py、tests/test\_urban\_long\_horizon.py。

## 4 能力与资源约束下的动态执行器路由

**作用** 在同能力执行器之间进行可解释选择，把运行历史反馈到排序中；失败后的重试、切换或重规划受恢复策略约束，而不是由模型自由决定。

---

### 算法 A4: 约束过滤、历史感知排序与有界恢复

---

**输入:** TaskNode  $v$ , 执行器注册表, 资源注册表, 历史统计, 已尝试集合与恢复预算

**输出:** 执行器与 RoutingDecision, 或无候选 / 恢复终止状态

- 1 扫描注册执行器, 保留 available、能力匹配且未被排除的候选;
  - 2 调用资源过滤函数检查位置、位置可用性、安全等级与配置的质量阈值等条件;
  - 3 按资源绑定和条件分支检查时延限制; 移除不通过候选;
  - 4 **if** 候选为空 **then**
  - 5   └ 抛出无可执行器异常, 由控制面处理;
  - 6 读取候选历史; 依据配置采用静态排序或历史感知 effective score;
  - 7 按评分及稳定次序排列候选, 选择首项;
  - 8 保存候选排名、评分分量、执行器 / 资源标识和排除集合;
  - 9 将执行器绑定到当前节点, 构造上下文并执行;
  - 10 记录成功、质量、Token 和耗时等反馈;
  - 11 **if** 节点执行失败 **then**
  - 12   └ 依据错误类型和策略, 选择同执行器重试、未尝试执行器切换、重规划或失败;
  - 13   └ 不增加 TaskSpec 允许的恢复预算; 上下文构造失败不走普通执行器切换;
  - 14 将业务结果交给独立验证路径; 执行返回成功不等于最终验收成功;
- 

**实现入口** orchestration/execution/capability\_registry.py: select\_executor、\_resource\_allowed; orchestration/execution/executor\_scoring.py; orchestration/graph/recovery\_controller.py: decide\_after\_failure。

**当前评分** 历史感知策略采用静态先验与运行历史的渐进混合:

$$s(e) = (1 - \lambda_e)s_{\text{static}}(e) + \lambda_e s_{\text{runtime}}(e) - p_{\text{failure}}(e), \quad \lambda_e = w_{\text{history}} \frac{N_e}{N_e + c}.$$

其中  $N_e$  是历史运行次数,  $c$  为冷启动参数; 分量来自质量、成本、时延、成功率和 Token 效率等。该式对应当前历史感知评分器; 静态策略则保留质量、成本、时延和执行器标识的稳定排序。

**复杂度** 令  $M$  为注册执行器数,  $C$  为资源条目数。当前资源过滤可能为每个执行器重复扫描注册资源, 因此选择成本的保守上界为:

$$T_{\text{route}} = O(MC + M \log(M + 1)), \quad S_{\text{route}} = O(M + C).$$

上述空间指候选与资源遍历临时结构, 不含历史库。仅在  $C$  有界或资源检查已索引化时, 才可把主要选择成本简化为  $O(M \log M)$ 。

**边界** 端、边、云为逻辑资源后端; 位置可用性检查不能替代物理资源逐项健康实测。当前决策记录包含合法候选等信息, 不宣称已完整记录所有被过滤资源的逐项淘汰原因。测试入口: tests/test\_dynamic\_routing.py、tests/test\_resource\_routing.py。

## 5 验证驱动的责任定位与局部重放

**作用** 把独立验证发现的问题定位到责任生产节点，重开责任节点及其后继，并保留未受影响节点的结果。业务修复预算不作为普通执行器重试预算的隐式延长。

---

### 算法 A5: 证据归因、受影响节点重开与有界再验证

---

**输入:** 当前图  $G$ , 业务验证结果, 证据核验问题, 最大修复轮数  $q$

**输出:** 无需修复、已修复、failed 或 blocked

```

1 从业务检查与证据核验结果构造带责任节点的问题集合  $I$ ;
2 记录阻断问题; 若验证通过且  $I = \emptyset$ , 返回无需修复;
3 if 存在已耗尽执行恢复的普通失败节点 then
4   | 返回 failed, 不额外授予业务重试;
5 for  $j \leftarrow 1$  to  $q$  do
6   | 从  $I$  提取可证明的责任节点集合  $O$ ;
7   | if  $O = \emptyset$  then
8   |   | 记录无法路由的问题; 返回 blocked;
9   |   构造修复上下文; 搜索  $O$  及其传递后继  $V'$ ;
10  |   重开  $V'$  中节点, 清除旧执行结果与执行器绑定; 保留其他节点状态;
11  |   登记本轮目标、影响范围和问题 ID, 调用正常调度器重放;
12  |   if 出现不可继续业务修复的执行失败 then
13  |     | 记录本轮失败; 返回 failed;
14  |   重新运行领域验证, 结合证据节点结果形成新问题集合  $I^+$ ;
15  |   依据验证证据关闭已消失问题, 保存本轮修复结果;
16  |   if 验证通过且  $I^+ = \emptyset$  then
17  |     | 返回已修复, 继续后续需求验收与最终门禁;
18  |   if 连续两轮无进展或预算耗尽 then
19  |     | 返回 blocked;
20  |    $I \leftarrow I^+$ ;

```

---

**实现入口** orchestration/graph/repair\_loop.py: BusinessRepairLoop.run; orchestration/graph/task\_graph.py: descendants\_of、reopen\_subgraph; orchestration/core/workflow.py: 修复后的全局验收衔接。

**范围与不变量** 当前采用传递后继闭包作为保守影响范围, 不宣称这是最小语义影响子图。范围之外的已完成结果不因本轮修复被清空; 范围之内必须重新产生可验证结果。修复函数返回 repaired 后, 仍需后续全局验收, 不能直接替代 Final Gate。

**复杂度** 当前 descendants\_of 反复扫描全图, 而不是从预先维护的后继邻接表直接遍历。设实际扫描轮数为  $h \leq n + 1$ , 则影响定位和重开节点的控制开销为:

$$T_{\text{affected}} = O(h(n + m) + n n') + T_{\text{topo}},$$

其中  $n' = |V'|$ ,  $n n'$  来自逐个重开节点时的线性查找;  $T_{\text{topo}}$  是全图拓扑排序成本。

若每轮还包含重放  $T_{\text{replay},j}$ 、验证  $T_{\text{validate},j}$  和归因 / 记录  $T_{\text{record},j}$ , 总时间必须按轮次求和。仅优化图搜索并不能消除模型和业务执行成本。测试入口: tests/test\_business\_repair\_loop.py、tests/test\_business\_validation.py。

## 6 复杂度分析的符号、假设与图控制面

**统一计量** 以下为代码结构推导的保守上界，不是实测耗时或紧确界。字典与集合操作按通常的期望常数时间计；节点标识、单个配置字段和单条历史摘要按长度有界计。可变长正文、JSON、artifact 必须另按字节规模计入；本地存储 I/O、模型调用及外部工具不藏入常数项。

| 符号                                  | 含义                              |
|-------------------------------------|---------------------------------|
| $n, m; n', m'$                      | 全图节点 / 边数；受影响子图节点 / 边数          |
| $M, C$                              | 注册执行器数；资源注册表条目数（两者不混用）          |
| $R, k, S_R$                         | 有效记忆条数；选取条数；记忆总内容规模             |
| $a_v$                               | 单条记忆评分中的图邻近度遍历工作量               |
| $S_K, K$                            | checkpoint 序列化规模；checkpoint 文件数 |
| $S_A, S_{\text{msg}}$               | 被哈希 artifact 字节数；消息实际内容规模       |
| $h, h_b, q$                         | 影响传播扫描轮数；失败阻塞传播轮数；修复轮数上限        |
| $T_{\text{topo}}, T_{\text{canon}}$ | 当前拓扑排序成本；JSON 规范化键排序成本          |

### 6.1 图合法性检查：线性理论界的适用条件

在邻接表、常数时间节点索引及 FIFO 队列下，标准 DAG 检查为  $O(n + m)$ 。当前 `validate_graph` 对每个节点 ID 调用列表 `count`，仅重复检查就需  $O(n^2)$ ；`topological_order` 还使用 `pop(0)`、候选队列重排序及子节点排序。因此，对当前实现采用：

$$T_{\text{topo}} = O(n^2 \log(n + 1) + m \log(n + 1))$$

作为保守上界。完整合法性检查还需计入输入输出契约、路径与 Schema 验证的内容成本。拓扑辅助结构为  $O(n + m)$ ；这一区分保留了标准算法理论界与当前确定性排序实现之间的差别。

### 6.2 就绪扫描：不只扫描节点，还要解析依赖

`get_node` 通过线性列表查找；一次全图依赖检查可达到  $O(nm + n)$ 。`ready_nodes` 先传播失败阻塞，再查询就绪状态并排序。在一次调用内，若阻塞传播经历  $h_b$  轮，则保守成本为：

$$T_{\text{ready}} = O((h_b + 1)(nm + n) + n \log(n + 1)).$$

因此，完整调度应按实际调用次数累加该成本，并计入状态保存与执行开销，不能仅因“每轮扫描所有节点”就统一写成  $O(n^2 + m)$ 。窗口化执行减少活动范围和部分持久化次数，但不自动消除这些全图查询。

### 6.3 受影响子图：执行范围小不等于定位开销只与子图有关

当前实现的影响传播与排序读取全图，A5 已给出相应成本。只有维护好后继索引、采用访问集合遍历，并使用常数时间节点定位时，局部搜索 / 重开的基础部分才可达到  $O(n' + m')$ ；若还要求稳定拓扑顺序，仍需另计排序成本。这些是可优化方向，不是本次提交宣称已经完成的性能保证。

## 7 其他模块、端到端成本与复核边界

| 模块            | 当前计量口径                      | 必要限定                                                     |
|---------------|-----------------------------|----------------------------------------------------------|
| 资源路由          | $O(MC + M \log(M + 1))$     | 资源位置与可用性查询可能重复扫描；不把资源数与执行器数混为一个变量。                       |
| 记忆构造          | 扫描、逐条评分、全候选排序，加查询与压缩        | 只写 $O(R)$ 仅覆盖扫描，遗漏排序、文本处理和图邻近度计算。                        |
| checkpoint 写入 | $O(S_K) + T_{\text{canon}}$ | JSON 编码、键排序、哈希及文件 I/O 分别计量。快照包含的结果越大， $S_K$ 越大。          |
| checkpoint 恢复 | 读取与规范化校验，加全图检查；必要时加文件排序     | 不把账本加载、完整图检查或业务重新验证当作常数时间。                               |
| artifact 哈希   | 时间 $O(S_A)$                 | 分块流式哈希的额外缓冲可有界；一次性读入或 JSON 重编码实现则需额外内容空间。                |
| 稀疏依赖消息        | 按实际依赖数与 $S_{\text{msg}}$ 计量 | 哈希、序列化、必需字段投影及预算调整都有内容成本；边数不等于字节数或 Token 数。              |
| 业务候选验证        | 按候选数与领域验证器计量                | 对 $P$ 个对象直接进行两两 AABB 检测，基础比较为 $O(P^2)$ ；不代表整个装箱求解器的总复杂度。 |

**持久化成本** 执行器统计按执行器 / 任务类型保存。若历史条目数为  $J$ 、序列化规模为  $S_J$ ，全量排序保存需额外考虑  $O(J \log(J + 1) + S_J)$ 。图快照、事件日志、上下文和 artifact 的实际写入次数也应分别统计；此开销不包含在单次候选排序中。

**端到端时间** 对于实际发生的调度、恢复和验收序列，应采用：

$$T_{\text{run}} = \sum T_{\text{control}} + \sum T_{\text{model}} + \sum T_{\text{tool}} + \sum T_{\text{validation}} + \sum T_{\text{I/O}}.$$

各项按实际执行路径计数，避免将模型时延解释为图算法性能，或将局部修复次数与全部运行成本混为一个指标。本附录不新增速度提升比例、不将千个 WorkUnit 等同于千次模型调用，也不宣称物理端边云集群已完成实测。

**代码与证据复核** A1–A5 均给出源码路径与测试入口；代码在提交系统源码包中，历史结果在运行证据中。普通回归测试与历史真实模型运行是不同证据类型；测试文件存在本身不是“本次全部测试通过”的声明。复杂度推导以本次核对的具体数据结构为准，不将标准算法的理想界直接替代工程实现界。

**改进方向** 节点 ID 索引、后继邻接表、增量 ready 队列、资源可用性索引与候选 Top- $k$  选择可分别降低查询、传播和排序成本。实施这些优化后应重新验证状态语义和验收结果，再更新复杂度，不以修改文档替代实现与测试。